

The Reliability Gap in Benchmark Auditing: Distribution Shift and Scale as Failure Modes of Contamination Detection

Wojciech Zarzecki^{1,2*} (✉), Jan Dubiński^{1,2}, and Sebastian Cygert^{1,3}

¹ NASK National Research Institute, Warsaw, Poland

² Warsaw University of Technology, Warsaw, Poland

wojciechzarzecki5@gmail.com

³ Gdańsk University of Technology, Gdańsk, Poland

Abstract. Benchmark contamination, where evaluation examples appear in a model’s training data, threatens the validity of LLM assessment. Statistical tools for detecting training-data membership exist, but have been validated almost exclusively in controlled academic regimes: large, homogeneous pre-training corpora and transparent, single-stage training pipelines. Whether these methods remain reliable in realistic auditing scenarios remains unclear. We identify two under-studied failure modes: distribution shift, which arises when suspect and validation sets violate the IID assumption, and scale constraints, which arise because benchmarks are orders of magnitude smaller than pre-training corpora. We systematically evaluate three leading paradigms: LLM Dataset Inference, Post-Hoc Dataset Inference, and CoDeC across 25 models from multiple families (including Pythia, OLMo 2, and specialised cultural and medical LLMs) and scales (up to 27B). We then further extend our analysis to frontier industry models. Across 335 evaluations, only 201 yield correct outcomes. LLM Dataset Inference results in false positives under distribution shift, Post-Hoc Dataset Inference is underpowered at benchmark scale, and CoDeC provides only coarse provenance signals that are insufficient to verify individual benchmark splits. Our results reveal a systematic reliability gap between controlled validation and practical benchmark auditing, and show that statistical detection cannot yet replace transparent data provenance. We open-source our benchmark for further research.

Keywords: benchmark contamination · dataset inference · LLM evaluation · distribution shift · training data auditing

1 Introduction

In the era of rapidly scaling large language models, both model size and training data volume have increased by orders of magnitude. This scaling has unlocked

* This work was conducted mostly while the author was at NASK National Research Institute.

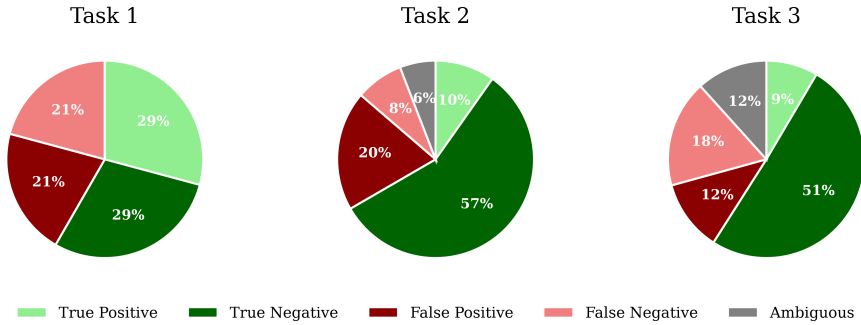


Fig. 1. Summary of our evaluation of methods for detecting whether a model was trained on a benchmark, across three tasks: Task 1 evaluates vulnerability to limited reference data (Table 1); Task 2 evaluates split-level benchmark exposure on instruction-tuned OLMo 2 (Table 2); and Task 3 evaluates specialised post-training datasets, for medical QA and Polish-language LLMs (Tables 3 and 4). The counts aggregate detection outcomes from these tables. Overall, only 201 out of 335 (60%) of the detection challenges are solved, suggesting that current approaches are not yet reliable for certifying benchmark integrity.

new capabilities in reasoning, mathematics, and multi-domain knowledge, pushing performance on many public benchmarks to saturation. At the same time, benchmark contamination has emerged as a serious threat to the validity of LLM evaluation.

When benchmark splits leak into scraped or aggregated corpora, benchmark performance may reflect prior exposure rather than genuine generalization. As models saturate widely used benchmarks, distinguishing between reasoning capability and memorization has become a central challenge of modern LLM assessment.

Existing literature addresses this issue by developing principled statistical tools for detecting training data, including *LLM Dataset Inference* [23], *Post-Hoc Dataset Inference* [31], and direct contamination measures such as *CoDeC* [30]. However, these methods are primarily validated in restricted academic settings, for example using the Pythia [3] suite trained on The Pile [12], where full data transparency and strict distributional assumptions hold. Academic model suites typically rely on relatively simple, single-stage pre-training on large, homogeneous corpora, which simplifies contamination analysis.

It remains unclear whether these methods remain reliable in more realistic settings. In practice, benchmark auditing concerns evaluation sets that are orders of magnitude smaller than pre-training corpora, and models that undergo multiple post-training stages, including instruction tuning on curated mixtures. To study this beyond academic model suites, our evaluation includes 21 openly available instruction-tuned models with sizes reaching 27B parameters (plus 4 pre-trained-only models, for 25 in total). Therefore, in this work, we investigate whether existing detection mechanisms function effectively *in the wild*. We

define this setting through two constraints: (1) the target of detection is a benchmark rather than a massive pre-training corpus, and (2) the audited models are instruction-tuned variants that have undergone multi-stage post-training.

Our main contributions are as follows. **First in-the-wild evaluation of benchmark auditing.** We provide the first systematic evaluation of three state-of-the-art detection paradigms, *LLM Dataset Inference*, *Post-Hoc Dataset Inference*, and *CoDeC*, beyond controlled academic corpora and in realistic benchmark auditing scenarios for which we open-source our code⁴.

Throughout our extensive evaluation, we find that:

1. **Current auditing methods are not yet reliable enough to consistently verify benchmark exposure.** As summarized in Figure 1, correct detection outcomes are matched by many failures across our evaluations.
2. **At limited dataset scale, only LLM DI works, but only under favorable conditions.** When the investigated dataset is as small as a typical benchmark, LLM DI can still detect training exposure if it has access to a genuinely unseen and approximately IID reference set. Post-Hoc DI does not have enough data to build a reliable synthetic reference set, and CoDeC assigns similar scores to closely matched train and test splits (Section 5.1).
3. **None of the methods can reliably tell which split of a benchmark was used for training.** In split-level auditing, LLM DI can mistake distribution differences for training exposure, Post-Hoc DI remains unstable, and CoDeC does not clearly separate seen and unseen splits from the same benchmark (Sections 5.2 and 5.5).
4. **For specialised post-training datasets and industry models, we find that the signals become less stable and harder to interpret.** For domain-specific post-training datasets, detection performance degrades. For industry models, the methods can detect differences at the level of model families. (Sections 5.3 and 5.4).

These findings suggest several practical recommendations. LLM DI is only usable when a genuinely unseen and approximately IID validation set is available. Post-Hoc DI appears less suitable for standard benchmark-sized datasets unless the quality of the synthetic held-out data can be independently assessed. CoDeC is best interpreted as a comparative contamination indicator rather than a certification tool. Our results suggest that transparent data provenance remains the most reliable basis for benchmark-integrity claims, with statistical auditing serving as complementary evidence.

2 Related Work

Membership and dataset inference. Membership inference attacks (MIAs) study whether a particular record was part of a model’s training set, typically by exploiting systematic differences in model behavior on seen versus unseen

⁴ <https://anonymous.4open.science/r/reliability-gap-benchmark-auditing/>

examples [28]. For large models, including LLMs, single-example MIAs can be noisy [23,11], motivating *dataset inference* methods that aggregate weak per-sample signals over a suspect set and compare them to an unseen reference set [23,10,20]. A key practical challenge for these tests is obtaining a reference or validation set that is truly unseen and sufficiently IID with the suspect set; violations of this assumption can lead to confounded detections.

Benchmark contamination. Benchmark contamination refers to the overlap between a benchmark dataset and a model’s training data [4]. This overlap may occur when benchmark train or test splits are directly included in pre-training corpora, or when near-duplicate or highly overlapping variants are present in large scraped mixtures [22]. In such cases, benchmark performance may reflect prior exposure rather than generalization to unseen data. As benchmarks are repeatedly reused and increasingly incorporated into post-training mixtures, contamination becomes both more likely and more difficult to detect [2].

In parallel to documenting contamination risk, recent efforts have focused on developing methods for detecting it. Retrieval-based and behavioral auditing approaches test whether benchmark items can be recovered from candidate corpora or elicited from the model itself [8,13]. More recently, performance-based methods have framed contamination detection as a statistical problem, asking whether unusually strong benchmark results are better explained by prior exposure than by genuine generalization [7]. Another recent direction is in-context contamination detection: CoDeC [30] measures how adding examples from the same dataset changes model confidence, using these shifts to produce an interpretable dataset-level contamination score. The method requires only gray-box access to token probabilities and assumes no prior knowledge of the training corpus, making it particularly attractive for practical benchmark auditing.

3 Methods

In our experiments, we use the following methods: Post-Hoc Dataset Inference, LLM Dataset Inference, and CoDeC, which are described in this section.

LLM Dataset Inference. This method [23] builds on membership inference attacks, but shifts the unit of analysis from *individual samples* to *datasets*. For LLMs, a single-sample MIA is often too noisy to be reliably useful: many sequences are “easy” (low loss) even if they were never seen during training, and the membership signal for any particular example becomes faint as models scale.

Dataset inference addresses this by *aggregating weak membership signals across many samples* and testing whether, in aggregate, a *suspect set* looks more train-like than an *unseen validation set*. Concretely, one computes a collection of MIA-derived scores (e.g., loss or perplexity-based and related features) for samples from both sets, learns a simple aggregation of these features, and then performs a statistical test, typically a one-sided *t*-test, on held-out samples to decide whether the suspect set exhibits systematically stronger membership evidence than the validation set.

The output of dataset inference is a **p -value**. Under the null hypothesis H_0 that the model was not trained on the suspect set, the observed separation between suspect and validation should be explainable by chance. A small p -value (e.g., $p < 0.1$ as in [23]) is interpreted as evidence against H_0 , that is, as a detection of training on the suspect dataset. Importantly, this p -value is only meaningful under a key assumption: the validation set must be unseen during training and IID with the suspect set. If the two sets differ in distribution, for example a benchmark train split versus its test split, the test can become confounded and may yield false positives.

Post-Hoc Dataset Inference. Post-hoc DI is designed to relax the practical requirement of LLM dataset inference for an unseen, IID validation set. Instead of requiring a naturally occurring validation set, post-hoc DI constructs a synthetic validation set that is intended to be distribution-matched to the suspect data. The procedure has two stages.

(1) *Held-out data generation.* Starting from a corpus of documents for a given dataset, the method segments documents into short snippets and splits them into two disjoint pools. A small generator model is trained with a causal language modeling objective on one pool. The second pool is converted into prefix-suffix pairs; the generator is then used to produce artificial suffixes conditioned on the prefixes. The resulting dataset contains real suffixes, which form the suspect set, and synthetic suffixes, which serve as the validation set. Both sets share surface-level distributional properties, but only the suspect set could contain memorized training data from the audited model.

(2) *Post-hoc calibration.* The method trains two classifiers to distinguish suspect versus synthetic validation examples: (i) a text-only classifier c_{text} , and (ii) a combined classifier c_{comb} that augments text with MIA-derived features such as loss statistics. The central statistical question is whether the MIA features provide additional separability beyond what is already achievable from text alone. Concretely, one computes per-example classification scores and performs a one-sided t -test to determine whether the combined classifier improves separation relative to the text-only baseline. The output is a **p -value** under the null hypothesis H_0 that MIA features do not add meaningful signal once text is accounted for. A small p -value, for example $p < 0.1$, is interpreted as evidence that the audited model was trained on the suspect distribution.

Context-based contamination signals. Complementary to likelihood- and classifier-based auditing, CoDeC (Contamination Detection via Context) detects dataset-level contamination by measuring how same-dataset in-context examples affect next-token likelihood [30]. In-context learning often improves predictions on unseen data, but can become unhelpful or harmful when memorization is triggered. This yields an interpretable contamination score defined as the fraction of samples for which in-context examples reduce model confidence.

The key observation is that in-context learning is typically helpful when the model has not internalized the target distribution. Adding a few examples provides dataset-specific cues such as format or vocabulary and improves next-

token likelihood. In contrast, if the model has already been trained on the dataset or a close variant, in-context examples provide little new information.

Operationally, for each sample x from a dataset $\mathcal{D}$, CoDeC compares the model’s average log-likelihood on x in two settings: (i) a baseline setting where the model predicts x directly, and (ii) an in-context setting where x is preceded by a small number of other samples from $\mathcal{D}$. The per-sample score is the log-likelihood difference $\Delta(x)$, defined as in-context minus baseline. The dataset **contamination score** is the fraction of samples for which $\Delta(x) < 0$. Higher values indicate that in-context learning is frequently unhelpful or harmful, which is consistent with stronger contamination.

4 Experimental setup

In this section, we describe the experimental setup used to evaluate contamination-detection methods across multiple model families, datasets, and auditing scenarios. We create a publicly available codebase which adapts each method to shared interface for benchmarking and detail the method configurations, audited models and benchmarks, and the notation used to interpret detection outcomes in the experiments that follow.

4.1 Detection Methods Configuration.

LLM DI: This method mirrors the MIA metrics, statistical test, outlier handling, and normalization strategy of Post-Hoc DI. Datasets are evenly split: set A computes metrics and trains a linear classifier using 250 samples per side, while set B evaluates the final t -test using at least 1 000 texts, the maximum number of texts per split evaluated by authors of methods [23].

Post-Hoc DI: We follow the original method as closely as possible in our benchmark-auditing setting. We subsample 2 000 texts per set and clip outliers symmetrically at 2.5% per tail. A **Meta-Llama-3.1-8B** model fine-tuned with LoRA generates 128-token suffixes for paraphrasing. Calibration relies on a GPT-2 baseline text classifier, while combined features are evaluated using a linear classifier trained for 1 000 epochs with Adam. MIA metrics are z -score normalized prior to training. We utilize perplexity, k -min-probs ($k=0.05$), reversed k -max-probs ($k=0.05$), and zlib ratio as our MIA metrics. We apply a one-sided independent samples t -test with a significance threshold of $p < 0.1$,⁵ taking the mean over 3 independent runs.

CoDeC: The contamination score is calculated as a binary per-sample classification, assigning a value of 1 if confidence drops when context is added. To avoid prompt artifacts, log-probabilities are averaged from the 10th token onward, we strictly utilize 1 randomly sampled context example — following the official description of CoDeC pipeline [30].

⁵ increased from $p < 0.05$ due to high p-values for members (where an ideal method should have very low p-values for members) and consistency with LLM DI

4.2 Audited Models and Datasets.

Open Model Suites: Fully open model suites enable rigorous contamination studies by allowing inspection of training data to support ground-truth membership claims. We evaluate the **Pythia suite**, known for its tight coupling with **The Pile** reference corpus. We also audit **OLMo 2**, which details its broad knowledge pre-training [24], rigorous post-training (incorporating datasets like *UltraFeedback* [6], *tulu3 sft math* [21], *Competition Math* [15], and *GSM8K* [5]), and extensive evaluation phases.

Evaluation Benchmarks: For the open-model experiments, we evaluate benchmark exposure on OLMo 2 using high-impact evaluation sets frequently audited for leakage as their train splits are widely available and exert strong optimization pressure. Representative benchmarks include **GSM8K** [5] (grade-school math word problems), **MATH** and **Competition Math** [15] (multi-step competition mathematics), **DROP** [9] (discrete reasoning over paragraphs), and **MMLU** [14] (multi-domain knowledge evaluation).

PLLM: A family of Polish language LLMs [19], which undergoes supervised fine-tuning using automatic and manual datasets, followed by preference-based alignment. We audit the Llama-PLLM-8B, PLLM-12B, and non-commercial PLLM-12B-nc variants.

Medical LLMs: We evaluate domain-specialised models fine-tuned on public medical QA benchmarks. Tested benchmarks include MedQA-USMLE [16], MedMCQA [25], PubMedQA [17], and MedExpQA [1]. Audited models include variants of MedGemma [27] (4B, 27B), Meditron3 [26] (8B, 9B), Meerkat [18] (7B, 8B), and Neeto-1.0-8B [29].

4.3 Notation for Detection Outcomes.

Throughout the tables in this section, we report detection outcomes using colored symbols to indicate correctness at a glance:

- $\textcolor{green}{+}$ (green $+$): *true positive*; the method correctly identifies that the model was trained on the given data.
- $\textcolor{green}{-}$ (green $-$): *true negative*; the method correctly determines that the model was not trained on the given data.
- $\textcolor{red}{+}$ (red $+$): *false positive*; the method incorrectly indicates that the model was trained on the data when it was not.
- $\textcolor{red}{-}$ (red $-$): *false negative*; the method fails to detect that the model was in fact trained on the data.

The detection decision ($+$ vs. $-$) is derived differently for each method. For *LLM Dataset Inference*, a detection ($+$) is declared when the one-sided t -test p -value falls below 0.1, following the threshold used in [23]. For *CoDeC*, a detection ($+$) is declared when the contamination score is higher than 0.8 indicating strong contamination, no detection ($-$) when contamination score is below 0.6, and inconclusive (?) when the score is between 0.6 and 0.8, as suggested in the original work [30]. Tables with numerical values instead of symbols can be found in the supplementary material.

5 Experiments

We first test whether current contamination detection methods remain effective when only benchmark-scale reference sets are available for inspection, rather than full pre-training corpora (Section 5.1). We then examine whether these methods can distinguish between seen and unseen splits within the same benchmark (Section 5.2). Next, we evaluate the most effective approaches on specialised post-training datasets (Section 5.3). Finally, we apply a comparative analysis on industry models (Section 5.4) and analyze the failure modes underlying these results (Section 5.5).

5.1 Task 1: Auditing with Limited Reference Data

A fundamental challenge in applying current detection methods “in the wild” is the vast disparity in data scale: while pre-training corpora typically encompass gigabytes of text, specific evaluation benchmarks often consist of only a few megabytes. To rigorously evaluate detection performance under these constrained conditions, we simulate the data scarcity characteristic of benchmark auditing. We find that Post-Hoc DI becomes ineffective when trained on only a few thousand examples, while CoDeC cannot distinguish split-level membership when train and test distributions are too similar. Only LLM DI, which requires access to an IID validation set, remains reliable at benchmark scale.

Task Setup. Instead of utilizing the full corpus, we restrict the reference data to $n = 2,000$ documents per dataset subsampled from various subsets of The Pile. We then evaluate the detection methods across multiple Pythia model sizes, effectively mimicking a scenario where the auditor has access to only a fraction of the data distribution, comparable to the size of a standard evaluation benchmark, rather than the massive datasets used in prior academic validations.

Table 1. Detection of training on train and test splits of the Pile (Limited Data). We reorganize the results into a grid to highlight consistency across diverse subsets. Symbols: $+$ = correctly detected as trained on, $-$ = correctly detected as *not* trained on, $+$ = false positive, $-$ = false negative, $?$ = uncertain signal.

[illegible]

Findings. As shown in Table 1, LLM Dataset Inference correctly distinguishes train from test splits across all model sizes, confirming that our implementation reproduces prior academic findings under controlled conditions. Post-Hoc Dataset Inference, however, does not reject the null hypothesis of non-membership in most cases. At benchmark scale, the generator trained on only 2,000 samples per subset is too weak to produce realistic synthetic validation data, leading to uninformative calibration statistics. This interpretation is supported by an analysis of a full-scale held-out generation setting at Section 5.5. CoDeC assigns high contamination scores to both splits when distributions are similar. As a result, high scores may reflect exposure to either the train split alone or both splits, indicating sensitivity to distributional overlap rather than precise membership.

5.2 Task 2: Detecting Split-Level Benchmark Exposure

In this task, we move from controlled Pythia/Pile simulations to realistic benchmark auditing on instruction-tuned OLMo 2, and assess if current methods can distinguish between seen and unseen splits within the same benchmark. We find that none of the methods can robustly distinguish seen from unseen splits once the setting shifts from corpus-level datasets to split-level benchmark exposure.

Task Setup. We utilize OLMo 2, whose post-training mixture explicitly includes the training split of GSM8K, while their test splits are reserved for evaluation. As controls for entirely unseen benchmark data, we include MMLU and DROP: although OLMo 2 is evaluated on these benchmarks, it was not trained on their training splits. Our objective is to verify if detection mechanisms can correctly flag the train splits of GSM8K and Competition Math as contaminated, while refraining from flagging their test splits or the entirely held-out benchmarks (MMLU and DROP).

Table 2. Detection of training on train and test splits for OLMo 2. Symbols: $+$ = correctly detected as trained on, $-$ = correctly detected as *not* trained on, $+$ = falsely detected as trained on (false positive), $-$ = missed detection (false negative), $?$ = uncertain signal. Test set of GSM8K was not large enough to run Post-Hoc DI.

OLMo 2 version	GSM8K			MMLU			DROP			Corr./Tot.
	Train		Test	Train		Test	Train		Test	
	1B	7B	13B	1B	7B	13B	1B	7B	13B	
LLM DI	$+$	$+$	$+$	$-$	$-$	$-$	$+$	$+$	$+$	12/18
Post-Hoc DI	$-$	$-$	$+$	n/a	n/a	n/a	$-$	$-$	$-$	13/18
CoDeC	$+$	$-$	$-$	$+$	$+$	$?$	$-$	$?$	$-$	9/18

Findings. The results in Table 2 reveal clear failure modes for each method. LLM Dataset Inference produces false positives on DROP and MMLU even though OLMo 2 was not trained on their train splits. One possible explanation

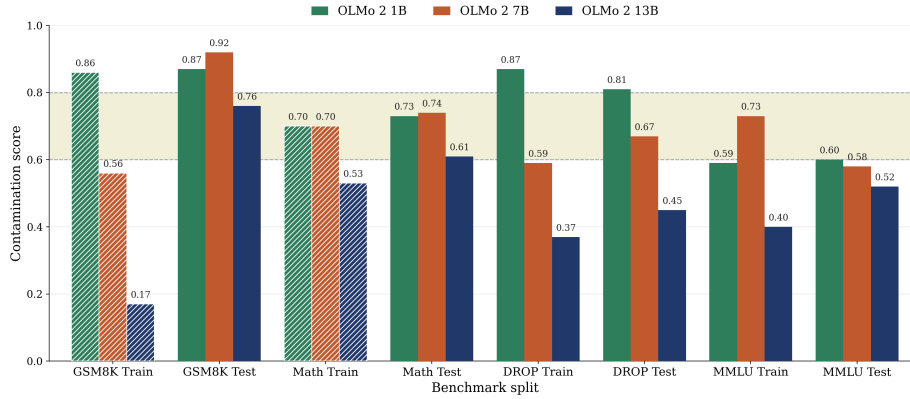


Fig. 2. CoDeC scores for Task 2. CoDeC contamination scores for benchmarks. Hatched bars indicate that this split was used as training split. We can observe trend between model sizes, however training splits do not have higher scores.

is that, although train and test splits of the same benchmark appear to be natural IID proxies, they can still differ in difficulty, style, or construction. In such cases, LLM DI can be driven by split-level distribution differences rather than by training exposure. Post-Hoc Dataset Inference again fails to cleanly separate train and test splits. As in Task 1, synthetic validation data introduces artifacts that dominate the statistical test. CoDeC assigns elevated contamination scores to GSM8K, consistent with its known post-training exposure; however, DROP and MMLU receive only slightly lower scores, so the signal does not provide clear evidence of split-level membership. Instead, the scores decrease with model size, a trend that Figure 2 reinforces by extending the same pattern to a further member benchmark, Competition Math.

5.3 Task 3: Auditing Specialised Post-Training Datasets

In this task, we examine whether contamination detection transfers from controlled benchmark settings to specialised post-training domains, specifically medical LLMs and mid-resource language models. We find that detection performance is highly inconsistent across these settings and deteriorates substantially when the available benchmark splits are small.

Task Setup. We apply LLM DI and CoDeC, the two methods that remained most effective in earlier experiments and were also the most practical to apply, to two downstream settings. For medical question answering, we evaluate MedGemma [27], Meditron3 [26], Meerkat [18], and Neeto-1.0 [29] on the train and test splits of MedQA-USMLE [16] and MedMCQA [25]. For Polish culture-aware language models, we evaluate the PLLuM-12B and Llama-PLLuM-8B families on train/test (or train/validation) splits from Automatic SFT, Manual SFT, and Alignment datasets. Because some held-out splits are very small, we

subsample the larger split to match the smaller one, yielding evaluation sets of approximately 1,600 examples for Automatic SFT and 1,150 for Manual SFT. This setting tests whether contamination signals remain informative on small, domain-specific post-training datasets.

Table 3. Detection of training on train and test splits for medical LLMs. Symbols: $+$ = correctly detected as trained on, $-$ = correctly detected as *not* trained on, $+$ = falsely detected as trained on (false positive), $-$ = missed detection (false negative), $?$ = uncertain signal.

		MedQA-USMLE		MedMCQA		Corr./Tot.
		Train	Test	Train	Test	
medgemma LLM DI	4B / 27B	- / -	- / -	- / +	- / -	5/8
	CoDeC	- / -	- / -	- / -	- / -	4/8
Meditron3 LLM DI	8B / 9B	- / -	- / -	+ / +	- / -	6/8
	CoDeC	? / -	? / -	- / -	- / -	3/8
meerkat LLM DI	7B / 8B	- / -	- / -	+ / -	- / -	5/8
	CoDeC	? / ?	? / ?	- / -	- / -	2/8
Neeto-1.0 LLM DI	8B	-	-	-	-	2/4
	CoDeC	-	-	-	-	2/4

Table 4. Detection of training on SFT and alignment data for PLLuM models. Symbols: $+$ = correctly detected as trained on, $-$ = correctly detected as *not* trained on, $+$ = falsely detected as trained on (false positive), $-$ = missed detection (false negative), $?$ = uncertain signal. base* = updated base checkpoint (250801).

		Automatic SFT		Manual SFT		Alignment		Corr./Tot.
		Train	Val	Train	Val	Train	Test	
PLLuM-12B base	base / base* / nc-base	- / -	- / -	- / -	- / -	+ / +	- / -	15/18
	LLM DI	+ / -	+ / -	? / -	? / -	- / -	- / -	14/18
PLLuM-12B SFT	chat / inst. / nc-chat / nc-inst.	+ / -	- / -	- / -	- / -	+ / +	- / -	15/24
	LLM DI	+ / +	+ / ?	? / ?	? / -	? / -	+ / -	9/24
Llama-PLLuM 8B base	base / base*	- / -	- / -	- / -	- / -	+ / +	- / -	10/12
	LLM DI	+ / -	+ / -	? / ?	+ / -	- / -	- / -	7/12
Llama-PLLuM 8B SFT	chat / inst.	- / -	- / -	- / -	- / -	+ / +	- / -	7/12
	LLM DI	+ / +	+ / +	+ / ?	+ / +	+ / -	+ / ?	5/12

Findings. The results in Tables 3 and 4 show that detection performance degrades substantially in downstream settings. For medical QA, LLM DI detects training on the MedMCQA train split for most models, though it misses it for medgemma-4B, meerkat-8B, and Neeto-1.0, and it consistently fails on MedQA-USMLE, missing the train split for every model. Held-out test splits are uniformly identified as non-members. CoDeC is less stable: it correctly identifies some train splits, but its behavior varies across models and does not consistently separate seen from unseen data.

For PLLuM, both methods are further limited by the small size of the evaluation splits. LLM DI often misses training on Automatic and Manual SFT, yielding repeated false negatives on train splits, while also producing false positives on Alignment. CoDeC shows partial success for the PLLuM-12B family, but performs substantially worse for Llama-PLLuM-8B and again tends to assign elevated scores to both train and held-out splits from the same dataset.

5.4 Task 4: Comparative Auditing of Industry Models

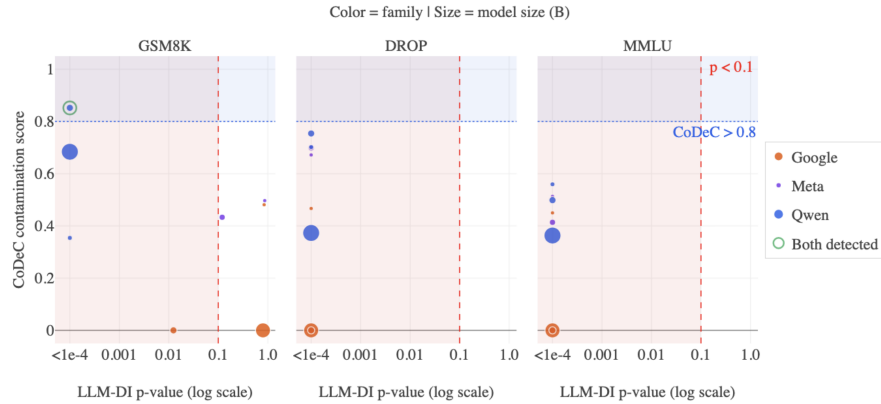


Fig. 3. Application of CoDeC and LLM Dataset Inference to industry models. We try to detect training on train splits of frontier benchmarks.

In this task, we extend the analysis to closed or partially disclosed industry models. We ask whether LLM DI and CoDeC provide consistent signals of prior exposure to benchmark data for frontier evaluations. We find that the signals are suggestive rather than definitive: most model families are flagged on at least some benchmarks, while Google’s Gemma-3 family stands out as the only series not flagged by LLM DI on GSM8K and having consistently lower CoDeC scores.

Task Setup. We evaluate LLM Dataset Inference and CoDeC on several industry model families — Google: Gemma 3 (1B, 4B, 12B); Alibaba: Qwen 3 (1.7B, 4B, 14B); Meta: Llama 3.2 (1B, 3B) — using public benchmarks: GSM8K, MMLU, and DROP. Because training data for these models is not fully disclosed, ground-truth membership is unavailable. Our goal is therefore comparative: to assess whether contamination signals differ systematically across model families.

Findings. As shown in Figure 3, most model families receive elevated CoDeC scores across benchmarks, and LLM DI frequently flags benchmark training. Gemma-3 is the main exception: it consistently shows lower CoDeC scores, and on GSM8K it is the only family not flagged by LLM DI on the train split. Based on this evidence, we infer that the Gemma-3 series was not likely trained on the GSM8K training set.

5.5 Diagnosis of Failure

The methods evaluated in previous tasks do not yield consistent outcomes. As shown in Figure 1, the number of successful detections is comparable to the number of failures; overall, just over 60% of the outcomes are correct. To understand these results, we examine the assumptions and failure modes of each method.

LLM Dataset Inference. LLM DI critically depends on the availability of a validation set that is both unseen and truly IID with the suspect dataset. This explains the contrast between Tasks 1 and later Tasks: LLM DI works in the controlled setting of Task 1. However, benchmark splits used in the later Tasks may break the IID assumption. These splits may differ in difficulty or construction, leading to the test capturing such differences rather than training exposure.

Post-Hoc Dataset Inference. On benchmark-sized subsets, Post-Hoc DI is dominated by the mismatch between real and synthetic held-out text rather than by memorization signal. As shown in Table 5, the text-only classifier is already highly discriminative, indicating strong separability between real and synthetic suffixes even without MIA features. In a well-calibrated Post-Hoc DI setup, MIA features should provide an additional discriminative signal beyond the text-only baseline. Here, however, adding MIA features yields only marginal improvements. For example, Common Crawl (410M) increases from 0.880 to 0.899, and Europarl (Train) slightly decreases from 0.895 to 0.890. This shows that the classification task is driven primarily by real-versus-synthetic distribution shift, not by training membership signal.

Calibration test p -values in Table 6 are correspondingly inconsistent. For Common Crawl (410M), train yields $p = 0.583$ while validation yields $p = 0.396$, both above the rejection threshold. Europarl produces $p \approx 1.000$ across splits and model sizes. Hacker News (Val) produces extremely small values such as $p = 0.007$ for 1.4B despite being a non-member split.

Table 5. Calibration-stage classifier AUCs, benchmark-scale subsampling. *Base* uses text only; *Comb* adds MIA features. Uniformly high Base AUCs (> 0.78) mean the real-vs-synthetic distribution shift alone is discriminative and MIA features barely help—the regime in which Post-Hoc DI fails. Cf. full-scale Table 7.

Dataset (Split)	Pythia 410M		Pythia 1.4B		Pythia 2.8B		Pythia 6.9B	
	<i>Base</i>	<i>Comb</i>	<i>Base</i>	<i>Comb</i>	<i>Base</i>	<i>Comb</i>	<i>Base</i>	<i>Comb</i>
CC (Train)	0.880	0.899	0.880	0.901	0.880	0.902	0.880	0.901
CC (Val)	0.882	0.899	0.882	0.900	0.882	0.902	0.882	0.900
Europarl (Train)	0.895	0.890	0.895	0.893	0.895	0.894	0.895	0.895
Europarl (Val)	0.887	0.890	0.887	0.891	0.887	0.893	0.887	0.893
Hacker News (Train)	0.899	0.911	0.899	0.912	0.899	0.915	0.899	0.914
Hacker News (Val)	0.899	0.928	0.899	0.929	0.899	0.929	0.899	0.930
Stack Exchange (Train)	0.788	0.804	0.788	0.803	0.788	0.805	0.788	0.803
Stack Exchange (Val)	0.832	0.844	0.832	0.844	0.832	0.845	0.832	0.844

Table 6. Calibration-test p -values, benchmark-scale subsampling ($n = 2,000$). Symbols: $+$ = correctly trained on ($p < 0.1$), $-$ = correctly *not* trained on, $+$ = false positive, $-$ = false negative. The test fails to reject the null for nearly every train split (12/32 correct), the failure mode caused by the distribution shift in Table 5. Cf. full-scale Table 8.

Dataset	Pythia 410M		Pythia 1.4B		Pythia 2.8B		Pythia 6.9B		Corr./Tot.
	Train	Val	Train	Val	Train	Val	Train	Val	
Pile CC	0.583 $-$	0.396 $-$	0.444 $-$	0.281 $-$	0.402 $-$	0.258 $-$	0.385 $-$	0.236 $-$	4/8
Europarl	1.000 $-$	1.000 $-$	1.000 $-$	0.999 $-$	1.000 $-$	0.999 $-$	1.000 $-$	0.999 $-$	4/8
Hacker News	0.450 $-$	0.049 $+$	0.441 $-$	0.007 $+$	0.315 $-$	0.006 $+$	0.388 $-$	0.009 $+$	0/8
Stack Exch.	0.709 $-$	0.821 $-$	0.707 $-$	0.811 $-$	0.700 $-$	0.755 $-$	0.747 $-$	0.788 $-$	4/8

Table 7. Calibration-stage classifier AUCs, full-scale held-out generation (authors’ released sets, generator trained on nearly the full subset). *Base* uses text only; *Comb* adds MIA features. Base AUCs fall to around 55% (e.g. Common Crawl Base: 0.548); real and synthetic suffixes become nearly indistinguishable and the distribution-shift artifact of Table 5 largely vanishes.

Dataset (Split)	Pythia 410M		Pythia 1.4B		Pythia 2.8B		Pythia 6.9B	
	<i>Base</i>	<i>Comb</i>	<i>Base</i>	<i>Comb</i>	<i>Base</i>	<i>Comb</i>	<i>Base</i>	<i>Comb</i>
CC (Train)	0.548	0.557	0.548	0.556	0.548	0.564	0.548	0.567
CC (Val)	0.506	0.513	0.506	0.515	0.506	0.517	0.506	0.523
Europarl (Train)	0.509	0.532	0.509	0.555	0.509	0.567	0.509	0.575
Europarl (Val)	0.514	0.516	0.514	0.523	0.514	0.526	0.514	0.533
Hacker News (Train)	0.525	0.567	0.525	0.578	0.525	0.585	0.525	0.591
Hacker News (Val)	0.570	0.569	0.570	0.571	0.570	0.578	0.570	0.577
Stack Exchange (Train)	0.540	0.554	0.540	0.562	0.540	0.567	0.540	0.574
Stack Exchange (Val)	0.518	0.523	0.518	0.533	0.518	0.534	0.518	0.538

Table 8. Calibration-test p -values (Diff), full-scale held-out generation (same subsets as Table 7). Symbols as in Table 6. With the distribution-shift artifact removed, the test aligns far better with ground truth than at benchmark scale (Table 6): 23/32 vs. 12/32 correct, with Hacker News and Europarl train splits now flagged on larger models, though Common Crawl and Stack Exchange remain undetected.

Dataset	Pythia 410M		Pythia 1.4B		Pythia 2.8B		Pythia 6.9B		Corr./Tot.
	Train	Val	Train	Val	Train	Val	Train	Val	
Pile CC	0.775 $-$	0.728 $-$	0.829 $-$	0.647 $-$	0.580 $-$	0.717 $-$	0.582 $-$	0.547 $-$	4/8
Europarl	0.184 $-$	0.882 $-$	0.000 $+$	0.675 $-$	0.000 $+$	0.696 $-$	0.000 $+$	0.350 $-$	7/8
Hacker News	0.002 $+$	0.744 $-$	0.007 $+$	0.958 $-$	0.000 $+$	0.774 $-$	0.002 $+$	0.876 $-$	8/8
Stack Exch.	0.598 $-$	0.830 $-$	0.254 $-$	0.435 $-$	0.216 $-$	0.377 $-$	0.116 $-$	0.247 $-$	4/8

To test it is this failure intrinsic to Post-Hoc DI, or just a symptom of limited data at benchmark scale, we re-run the calibration stage on the authors’ released held-out sets, whose generator is trained on nearly the full Pile subsets. Text-only AUCs collapse to around 55% (Table 7, e.g. Common Crawl Base: 0.548), so the distribution shift that dominated the benchmark-scale setting (Table 5) largely vanishes. With the artifact gone, the calibration test recovers, rising from 12/32

to 23/32 correct p -values (Table 8 vs. Table 6). Post-Hoc DI is underpowered at benchmark scale, working only with enough data to train a well-matched synthetic generator.

CoDeC. CoDeC exhibits a different limitation than the DI-based methods: it preserves broad provenance differences, but lacks the resolution needed for split-level auditing. In Figure 4, contamination scores for known pre-training corpora are substantially higher than for evaluation-only benchmarks, reproducing prior findings. However, train and test splits within the same dataset yield only marginal differences, indicating difficulty distinguishing near-IID conditions.

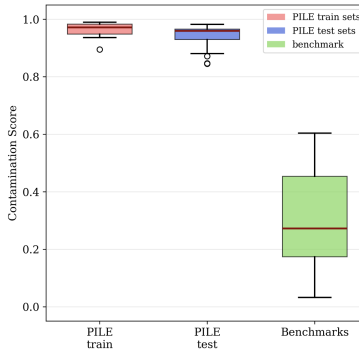


Fig. 4. CoDeC contamination scores on Pythia. Evaluation-only benchmarks score lower than pre-training corpora (reproducing prior work), but train and test splits of the same corpus score nearly identically.

In instruction-tuned models (Figure 5), aggregation by dataset provenance reveals a coarse ordering: pre-training sources receive the highest scores, post-training datasets are intermediate, and evaluation-only benchmarks are lowest. At the level of individual datasets and model sizes, however, the scores remain variable and overlapping. CoDeC is therefore best interpreted as a coarse contamination indicator rather than a precise split-level detector. Used comparatively—many models on one dataset, flagging outliers—it still offers a useful relative signal of likely contamination.

6 Limitation and Future Work

Our study has several limitations that also point to future work. Our results come from configurations run under a shared interface with default hyperparameters rather than exhaustive per-task tuning, which might otherwise improve the effectiveness of the evaluated methods. Our open-weight audit reaches 27B parameters and centres on math, QA, and knowledge benchmarks in English, with additional Polish (PLLuM) and medical models; larger open models, other

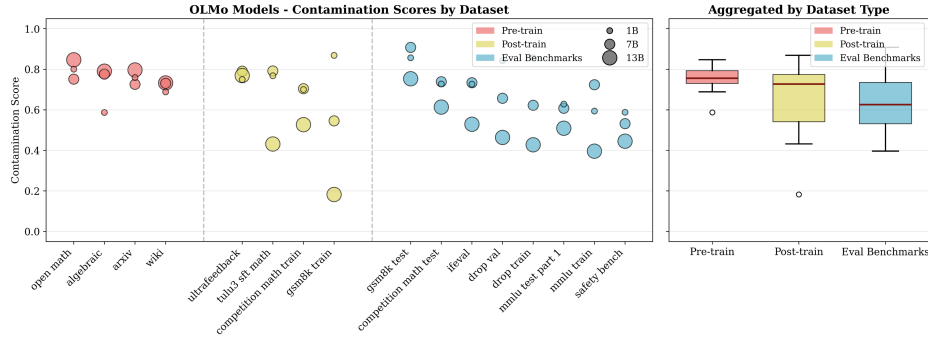


Fig. 5. CoDeC scores for OLMo 2 (instruction-tuned) grouped by data provenance. Pre-training sources receive the highest contamination scores, post-training datasets used during instruction tuning are intermediate, and evaluation-only benchmarks score lowest. This coarse ordering is consistent with known data membership, but the overlap between adjacent groups limits the method’s utility for certifying individual benchmark integrity.

languages and modalities, and free-form generation benchmarks are not covered, and behaviour may differ at frontier scale. For closed industry models we lack verified membership, so that evidence is comparative across model families rather than definitive. Finally, we characterise *when* and *why* current methods fail in the wild, but do not propose a new detector that overcomes these failures. Quantifying how much distribution shift or how little data each method can tolerate, and designing auditing tools robust to both, remains open.

7 Conclusions

In this work, we examined the reliability of contamination detection methods by moving from the controlled settings of prior literature to the realistic, "in-the-wild" regime of modern instruction-tuned models. We stress-tested three leading detection paradigms against the complexities of post-training mixtures and opaque data provenance. Our findings reveal that the transition from academic validation to practical auditing is fraught with challenges:

- **The I.I.D. assumption is a critical vulnerability.** We found that *LLM Dataset Inference* [23] is highly sensitive to distribution shifts. While effective when a perfect I.I.D. validation set exists, it yields significant false positives when using standard benchmark test splits as validation proxies (as seen with DROP and MMLU). This makes it dangerous to use as a sole arbiter for leaderboard integrity without verifying distributional alignment.
- **Benchmark-scale datasets are too small for generative methods.** While *Post-Hoc Dataset Inference* [31] theoretically removes the need for natural validation data, we showed it is impractical for standard benchmarks.

Unlike massive pre-training corpora, benchmarks like GSM8K lack the text volume required to effectively train the required generators.

- **Detection granularity is limited.** *CoDeC* [30] provides a valuable coarse-grained signal—effectively separating pre-training sources from held-out tasks in aggregate. However, its high variance at the individual dataset train level (Figure 5) limits its utility for certifying specific benchmark results.

We conclude that there is currently no "silver bullet" for detecting contamination in the wild. Consequently, statistical auditing cannot yet fully replace **transparent data provenance**.

Acknowledgments. This research was supported by the Polish National Science Centre (NCN) within grants no. 2025/57/N/ST6/04025. We gratefully acknowledge Polish high-performance computing infrastructure PLGrid, HPC Center: ACK Cyfronet AGH, for providing computer facilities and support within computational grant no. PLG/2024/017781 and PLG/2025/018634.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Alonso, I., Oronoz, M., Agerri, R.: Medexpqa: Multilingual benchmarking of large language models for medical question answering. ArXiv **abs/2404.05590** (2024)
2. Balloccu, S., Schmidová, P., Lango, M., Dušek, O.: Leak, cheat, repeat: Data contamination and evaluation malpractices in closed-source llms. ArXiv **abs/2402.03927** (2024)
3. Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O’Brien, K., et al.: Pythia: A suite for analyzing large language models across training and scaling. ArXiv **abs/2304.01373** (2023)
4. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., et al.: Language models are few-shot learners. ArXiv **abs/2005.14165** (2020)
5. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., et al.: Training verifiers to solve math word problems. ArXiv **abs/2110.14168** (2021)
6. Cui, G., Yuan, L., Ding, N., Yao, G., Zhu, W., et al.: Ultrafeedback: Boosting language models with high-quality feedback. ArXiv **abs/2310.01377** (2023)
7. Dekoninck, J., Müller, M.N., Vechev, M.: Constat: Performance-based contamination detection in large language models. ArXiv **abs/2405.16281** (2024)
8. Deng, C., Zhao, Y., Tang, X., et al.: Investigating data contamination in modern benchmarks for large language models. ArXiv **abs/2311.09783** (2024)
9. Dua, D., Wang, Y., Dasigi, P., Stanovsky, G., Singh, S., et al.: DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. ArXiv **abs/1903.00161** (2019)
10. Dubiński, J., Kowalczyk, A., Boenisch, F., Dziedzic, A.: Cdi: Copyrighted data identification in diffusion models. In: Proceedings of the Computer Vision and Pattern Recognition Conference. pp. 18674–18684 (2025)

11. Dubiński, J., Kowalczyk, A., Pawlak, S., Rokita, P., Trzciński, T., Morawiecki, P.: Towards more realistic membership inference attacks on large diffusion models. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision. pp. 4860–4869 (2024)
12. Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., et al.: The pile: An 800gb dataset of diverse text for language modeling. ArXiv **abs/2101.00027** (2020)
13. Golchin, S., Surdeanu, M.: Time travel in llms: Tracing data contamination in large language models. ArXiv **abs/2308.08493** (2024)
14. Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., et al.: Measuring massive multitask language understanding. ArXiv **abs/2009.03300** (2021)
15. Hendrycks, D., Burns, C., Kadavath, S., Arora, A., et al.: Measuring mathematical problem solving with the MATH dataset. ArXiv **abs/2103.03874** (2021)
16. Jin, D., Pan, E., Oufattole, N., Weng, W.H., Fang, H., et al.: What disease does this patient have? a large-scale open domain question answering dataset from medical exams. ArXiv **abs/2009.13081** (2020)
17. Jin, Q., Dhingra, B., Liu, Z., Cohen, W., Lu, X.: PubMedQA: A dataset for biomedical research question answering. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP) (Nov 2019)
18. Kim, H., Hwang, H., Lee, J., Park, S., Kim, D., et al.: Small language models learn enhanced reasoning skills from medical textbooks. ArXiv **abs/2404.00376** (2024)
19. Kocoń, J., Piasecki, M., Janz, A., Ferdinan, T., Łukasz Radliński, et al.: Pllum: A family of polish large language models. ArXiv **abs/2511.03823** (2025)
20. Kowalczyk, A., Dubiński, J., Boenisch, F., Dziedzic, A.: Privacy attacks on image autoregressive models. arXiv preprint arXiv:2502.02514 (2025)
21. Lambert, N., Morrison, J.D., Pyatkin, V., Huang, S., et al.: Tülu 3: Pushing frontiers in open language model post-training. ArXiv **abs/2411.15124** (2024)
22. Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., et al.: Deduplicating training data makes language models better. ArXiv **abs/2107.06499** (2022)
23. Maini, P., Jia, H., Papernot, N., Dziedzic, A.: Llm dataset inference: Did you train on my dataset? ArXiv **abs/2406.06443** (2024)
24. OLMo, T., Walsh, P., Soldaini, L., Groeneveld, D., Lo, K., et al.: Olmo 2: Furious. ArXiv **abs/2501.00656** (2025)
25. Pal, A., Umapathi, L.K., Sankarasubbu, M.: Medmcqa : A large-scale multi-subject multi-choice dataset for medical domain question answering. ArXiv **abs/2203.14371** (2022)
26. Sallinen, A., Solergibert, A.J., Zhang, M., Boyé, G.B., et al.: Llama-3-meditron: An open-weight suite of medical LLMs based on llama-3.1. In: Workshop on Large Language Models and Generative AI for Health at AAAI 2025 (2025)
27. Sellergrén, A., Kazemzadeh, S., Jaroensri, T., Kiraly, A., Traverse, M., et al.: Medgemma technical report. ArXiv **abs/2507.05201** (2025)
28. Shokri, R., Stronati, M., Song, C., Shmatikov, V.: Membership inference attacks against machine learning models. ArXiv **abs/1610.05820** (2017). <https://doi.org/10.1109/SP.2017.41>
29. Verma, S.: Neeto: A specialized medical llm for neet-pg/ukmle/usmle preparation (2025), <https://huggingface.co/S4nfs/Neeto-1.0-8b>
30. Zawalski, M., Boubdir, M., Bałazy, K., Nushi, B., Ribalta, P.: Detecting data contamination in llms via in-context learning. ArXiv **abs/2510.27055** (2025)
31. Zhao, B., Maini, P., Boenisch, F., Dziedzic, A.: Unlocking post-hoc dataset inference with synthetic data. ArXiv **abs/2506.15271** (2025)

A Detailed Method Overviews

This section provides step-by-step procedural summaries for the three detection paradigms evaluated in the main text.

LLM Dataset Inference LLM Dataset Inference [23] detects training-data membership at the *dataset* level by aggregating weak per-sample membership inference signals and applying a statistical test. The procedure consists of four stages.

Stage 0 (Data preparation). The auditor requires two datasets drawn from the same distribution: a *suspect set* $\mathcal{D}_{\text{sus}}$, hypothesised to have been used for training, and a *validation set* $\mathcal{D}_{\text{val}}$, known to be unseen by the model. Both sets are randomly partitioned into non-overlapping A and B splits.

Stage 1 (Feature extraction). For every sample in the A-partitions, a battery of membership inference attacks (MIAs) is executed against the suspect LLM f_θ to produce a feature vector. These features span loss-based metrics (perplexity, Min- $k\%$), perturbation-based scores (synonym substitution, character-level edits), reference-model comparisons, and compression-based ratios—52 scores in total.

Stage 2 (Correlation learning). A linear regressor is trained on the A-split features, assigning label 0 to suspect samples and label 1 to validation samples. The learned weights identify which MIA features carry positive membership signal for the given data distribution, since no single MIA works across all distributions.

Stage 3 (Statistical test). The regressor produces per-sample scores on the held-out B-splits. A one-sided t -test assesses whether the suspect-set scores are systematically lower (more member-like) than the validation-set scores, yielding a p -value under the null hypothesis H_0 that the suspect set was not used for training.

Critical assumption. The validation set must be both IID with the suspect set and genuinely unseen during training. When this assumption is violated—for example when using a benchmark test split as validation for its train split—the method can produce false positives, as demonstrated in Task 2 of our evaluation.

Post-Hoc Dataset Inference Post-Hoc Dataset Inference [31] removes the requirement for a natural IID validation set by synthetically generating one. The method operates in two phases.

Phase 1: Held-out data generation. The suspect documents are segmented into short text snippets, shuffled, and randomly split into a *generator training set* and a *generator inference set*. A small causal language model is fine-tuned (via LoRA) on the training set using a next-token prediction objective. Each snippet in the inference set is then split into a prefix and a suffix. The trained generator produces synthetic suffixes conditioned on the prefixes. The original (real) suffixes constitute the suspect set $\mathcal{D}_{\text{sus}}$; the synthetic suffixes con-

stitute the held-out set $\mathcal{D}_{\text{val}}$. Because every real-synthetic pair shares a common prefix, the two sets are positionally aligned, minimising distributional shift.

Phase 2: Post-hoc calibration. Both sets are split into train and test subsets. Two classifiers are trained on the training subsets: (i) a *text-only classifier* $c_{\text{text}}(x)$ that distinguishes real from synthetic text based on textual features alone, and (ii) a *combined classifier* $c_{\text{comb}}(x, \text{MIA}(f(x)))$ that additionally incorporates MIA-derived features from the suspect LLM f . The key insight is that any performance advantage of c_{comb} over c_{text} must stem from genuine membership signals rather than the real-vs-synthetic distributional shift. A one-sided t -test on the test subsets checks whether this advantage is statistically significant; rejection of H_0 indicates that the suspect set was used for training.

Critical assumption. The generator must be trained on sufficient data to produce realistic synthetic text. When trained on only $\sim 2,000$ benchmark-sized samples, the generator is too weak: the text-only classifier achieves high AUC, the real-vs-synthetic gap dominates, and the calibration test becomes uninformative.

CoDeC: Contamination Detection via Context CoDeC [30] detects dataset-level contamination by measuring how in-context examples from the same dataset affect the model’s prediction confidence. The procedure operates on each sample independently before aggregating.

Step 1 (Baseline prediction). For each sample $x \in \mathcal{D}$, compute the model’s average per-token log-likelihood on x without any preceding context.

Step 2 (In-context prediction). Sample n additional examples from $\mathcal{D} \setminus \{x\}$, prepend them to x as in-context demonstrations, and recompute the model’s average log-likelihood on x in this extended context.

Step 3 (Score computation). Compute the per-sample confidence difference $\Delta(x) = \text{logprob}_{\text{ICL}}(x) - \text{logprob}_{\text{baseline}}(x)$. For unseen data, in-context examples typically improve confidence ($\Delta > 0$), since they provide useful distributional cues such as format, vocabulary, and style. For memorised data, context can disrupt memorisation patterns, *reducing* confidence ($\Delta < 0$).

Step 4 (Aggregation). The dataset contamination score is

$$S(\mathcal{D}) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\Delta(x_i) < 0],$$

the fraction of samples for which context reduces confidence. Scores above 80% indicate strong contamination evidence; scores below 60% suggest no contamination; intermediate values require cross-model comparison for reliable interpretation.

Key properties. CoDeC requires no external reference data, held-out sets, or dataset-specific calibration. It produces percentage-based scores that are directly interpretable and model-agnostic, needing only gray-box access to token log-probabilities. The method is also computationally efficient, requiring only two forward passes per sample.

B Numerical values for evaluation experiments

We present numerical values which stand for symbols in tables in main text.

Table 9. Numeric contamination-detection scores on train and test splits of the Pile (Limited Data). LLM DI and Post-Hoc DI entries are p-values; CoDeC entries are contamination scores.

Pythia sizes	Pile CC						Pile Europarl									
	Train			Test			Train			Test						
	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B					
LLM DI	1.00	/ 1.00	/ 0.95	/ 1.00	1.00	/ 1.00	/ 1.00	/ 1.00	0.00	/ 0.00	/ 0.00	1.00	/ 1.00	/ 1.00	/ 1.00	
Post-Hoc DI	0.58	/ 0.44	/ 0.40	/ 0.39	0.40	/ 0.28	/ 0.26	/ 0.24	1.00	/ 1.00	/ 1.00	1.00	/ 1.00	/ 1.00	/ 1.00	
CoDeC	0.88	/ 0.86	/ 0.87	/ 0.86	0.84	/ 0.84	/ 0.84	/ 0.86	0.92	/ 0.94	/ 0.96	/ 0.97	0.96	/ 0.97	/ 0.97	/ 0.98
Pythia sizes	Pile Hacker News						Pile Stack Exchange									
	Train			Test			Train			Test						
	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B	0.4B/1.4B/2.8B/6.9B					
LLM DI	0.01	/ 0.01	/ 0.02	/ 0.00	1.00	/ 1.00	/ 0.98	/ 1.00	0.00	/ 0.00	/ 0.01	/ 0.00	1.00	/ 1.00	/ 1.00	/ 1.00
Post-Hoc DI	0.45	/ 0.44	/ 0.32	/ 0.39	0.05	/ 0.01	/ 0.01	/ 0.01	0.71	/ 0.71	/ 0.70	/ 0.75	0.82	/ 0.81	/ 0.75	/ 0.79
CoDeC	0.90	/ 0.95	/ 0.97	/ 0.97	0.91	/ 0.94	/ 0.95	/ 0.96	0.94	/ 0.96	/ 0.95	/ 0.95	0.88	/ 0.87	/ 0.85	/ 0.84

Table 10. Numeric contamination-detection scores for OLMo 2. LLM DI and Post-Hoc DI entries are p-values; CoDeC entries are contamination scores.

	GSM8K						MMLU						DROP					
	Train			Test			Train			Test			Train			Test		
	1B	/ 7B	13B				1B	/ 7B	13B				1B	/ 7B	13B			
<i>OLMo 2 version</i>																		
LLM DI	0.00 / 0.00	/ 0.00	1.00 / 1.00	/ 1.00	/ 1.00	0.00 / 0.00	/ 0.00	/ 0.00	1.00 / 0.97	/ 0.99	0.00 / 0.00	/ 0.00	/ 0.00	1.00 / 1.00	/ 1.00	/ 0.51		
Post-Hoc DI	0.80 / 0.54	/ 0.06	n/a / n/a	/ n/a	/ n/a	0.68 / 0.75	/ 0.61	0.74 / 0.90	/ 0.76	1.00 / 1.00	/ 1.00	/ 0.99	/ 0.99	/ 0.99	/ 0.99	/ 0.45		
CoDeC	0.86 / 0.56	/ 0.17	0.87 / 0.92	/ 0.76	0.59 / 0.73	/ 0.40	0.60 / 0.58	/ 0.52	0.87 / 0.59	/ 0.37	0.81 / 0.67	/ 0.49						

Table 11. Contamination detection scores for medical LLMs (numeric values).

		MedQA-USMLE		MedMCQA	
		Train	Test	Train	Test
medgemma	LLM DI	0.17 / 0.25	1.00 / 0.73	0.17 / 0.01	1.00 / 0.13
4B / 27B	CoDeC	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00
Meditron3	LLM DI	0.69 / 0.98	0.63 / 0.83	0.00 / 0.05	1.00 / 1.00
8B / 9B	CoDeC	0.66 / 0.29	0.64 / 0.31	0.37 / 0.32	0.34 / 0.28
meerkat	LLM DI	0.48 / 0.99	1.00 / 0.97	0.03 / 0.26	1.00 / 1.00
7B / 8B	CoDeC	0.71 / 0.65	0.71 / 0.63	0.47 / 0.45	0.46 / 0.48
Neeto-1.0	LLM DI	0.83	0.88	0.73	1.00
8B	CoDeC	0.52	0.51	0.43	0.48

Table 12. Contamination detection scores for PLLuM models on SFT data (numeric values). base* = updated base checkpoint (250801).

Automatic SFT			
		Train	Val
PLLuM-12B base	<i>base / base* / nc-base</i>		
	LLM DI	0.92 / 0.62 / 0.85	0.71 / 0.26 / 0.90
	CoDeC	0.85 / 0.50 / 0.38	0.82 / 0.49 / 0.37
PLLuM-12B SFT	<i>chat / inst.</i>		
	LLM DI	0.00 / 0.32	0.50 / 0.74
	CoDeC	0.90 / 0.89	0.86 / 0.89
	<i>nc-chat / nc-inst.</i>		
	LLM DI	0.46 / 0.81	0.18 / 0.16
	CoDeC	0.81 / 0.61	0.79 / 0.59
Llama-PLLuM 8B base	<i>base / base*</i>		
	LLM DI	0.64 / 0.15	0.72 / 0.42
	CoDeC	0.95 / 0.55	0.96 / 0.57
Llama-PLLuM 8B SFT	<i>chat / inst.</i>		
	LLM DI	0.23 / 0.96	0.48 / 0.74
	CoDeC	0.93 / 0.94	0.92 / 0.93
Manual SFT			
		Train	Val
PLLuM-12B base	<i>base / base* / nc-base</i>		
	LLM DI	0.30 / 0.34 / 0.80	0.62 / 0.22 / 0.31
	CoDeC	0.65 / 0.54 / 0.48	0.69 / 0.52 / 0.50
PLLuM-12B SFT	<i>chat / inst.</i>		
	LLM DI	0.91 / 0.82	0.39 / 0.29
	CoDeC	0.78 / 0.66	0.79 / 0.77
	<i>nc-chat / nc-inst.</i>		
	LLM DI	0.91 / 0.51	0.27 / 0.60
	CoDeC	0.80 / 0.59	0.81 / 0.64
Llama-PLLuM 8B base	<i>base / base*</i>		
	LLM DI	0.82 / 0.65	0.64 / 0.26
	CoDeC	0.75 / 0.60	0.83 / 0.57
Llama-PLLuM 8B SFT	<i>chat / inst.</i>		
	LLM DI	0.91 / 0.80	0.65 / 0.23
	CoDeC	0.84 / 0.74	0.90 / 0.81

Table 13. Contamination detection scores for PLLuM models on alignment data (numeric values). base* = updated base checkpoint (250801).

Alignment			
		Train	Test
PLLuM-12B base	<i>base / base* / nc-base</i>		
	LLM DI	0.00 / 0.00 / 0.00	0.17 / 0.67 / 0.96
	CoDeC	0.28 / 0.39 / 0.25	0.43 / 0.45 / 0.29
PLLuM-12B SFT	<i>chat / inst.</i>		
	LLM DI	0.00 / 0.00	0.85 / 0.21
	CoDeC	0.80 / 0.39	0.79 / 0.56
	<i>nc-chat / nc-inst.</i>		
	LLM DI	0.00 / 0.00	0.94 / 0.81
	CoDeC	0.85 / 0.42	0.84 / 0.51
Llama-PLLuM 8B base	<i>base / base*</i>		
	LLM DI	0.00 / 0.00	0.12 / 0.68
	CoDeC	0.37 / 0.36	0.47 / 0.47
Llama-PLLuM 8B SFT	<i>chat / inst.</i>		
	LLM DI	0.00 / 0.00	0.97 / 0.84
	CoDeC	0.83 / 0.49	0.85 / 0.64

C Time analysis

Table 14. Average run duration for detection methods. Standard deviation is reported across different model sizes (Pythia, OLMo 2) and datasets (Pile subsets, GSM8K, DROP and MMLU). All values are normalized to seconds for comparability. The runs were executed on Nvidia A100 Nvidia GH200.

Method	Average Duration [s]
Post-Hoc DI	4566.6 $\pm$ 846.4
CoDeC	206.4 $\pm$ 232.6
LLM DI	748.1 $\pm$ 447.4